

LAB EXERCISE 1

Study of System Design Concepts and Development Tools

Team Members

S.No	Name	Register Number
1	Dinesh Kumar J	3122245001042
2	Gopinath S	3122245001048
3	Harish R	3122245001055
4	Srikrishna O S	3122245001312

AIM:

To understand the fundamentals of system design and the architecture of scalable distributed systems, to familiarize with the software tools used in the System Design Laboratory, and to learn the role of each tool in designing, developing, testing, and deploying distributed applications.

Task 1: Fundamentals of System Design

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy specified functional and non-functional requirements.

For large-scale distributed applications, the key properties considered are:

- Scalability – the ability of a system to handle increasing load by adding resources, either vertically (more powerful machines) or horizontally (more machines).
- Availability – the proportion of time a system remains operational and accessible, typically achieved through redundancy and failover mechanisms.
- Reliability – the ability of a system to continue performing its intended function correctly even in the presence of failures.
- Consistency – ensuring that all nodes in a distributed system see the same data at the same time, governed in practice by trade-offs described by the CAP theorem (Consistency, Availability, Partition tolerance – a system can only fully guarantee two of the three at once).
- Latency and Throughput – latency is the time taken to process a single request, while throughput is the number of requests a system can process in a given time.
- Load Balancing – distributing incoming traffic across multiple servers to prevent any single server from becoming a bottleneck.
- Caching – storing frequently accessed data in fast-access storage to reduce latency and load on the primary data store.
- Database Sharding and Replication – sharding splits data across multiple databases to distribute load, while replication copies data across nodes for redundancy and read scalability.

Software Tools Used in the Laboratory

VS Code (Apache VS Code IDE): A lightweight, extensible source-code editor used to write, run, and debug code in multiple languages, with built-in Git integration.

Java: A general-purpose, object-oriented programming language used to implement the backend services and prototypes built during the lab.

JMeter: An open-source load and performance testing tool used to simulate concurrent users and measure how a system behaves under load.

Postman: An API development and testing tool used to send requests to a service and inspect responses, useful for verifying REST APIs during development.

MongoDB: A NoSQL document-oriented database used to store data in flexible, JSON-like documents, well suited for horizontally scalable systems.

Kafka: A distributed event-streaming platform used to publish, subscribe to, and process streams of records in real time, commonly used for decoupling services.

GitHub: A cloud-based hosting platform for Git repositories, used for version control, collaboration, and code review among team members.

Redis: An in-memory key-value data store used primarily as a cache to reduce database load and latency for frequently accessed data.

Nginx: A high-performance web server and reverse proxy, commonly used as a load balancer and entry point in front of multiple backend service instances.

Task 2: Comparison of Software Tools

The table below summarizes the category and role each tool plays in developing scalable distributed systems.

Tool	Category	Role in Scalable Distributed Systems
VS Code	Development Environment	Code editing, debugging, and Git integration for building services.
Java	Programming Language	Implements application logic and backend services for distributed systems.
JMeter	Performance Testing	Simulates load to evaluate scalability and identify bottlenecks.
Postman	API Testing	Verifies correctness of REST APIs exposed by distributed services.
MongoDB	NoSQL Database	Provides horizontally scalable, schema-flexible data storage.
Kafka	Message Broker / Event Streaming	Decouples services and enables asynchronous, fault-tolerant communication.
GitHub	Version Control	Enables collaborative development, code review, and change tracking.
Redis	In-Memory Cache	Reduces latency and database load via fast in-memory data access.

Tool	Category	Role in Scalable Distributed Systems
Nginx	Web Server / Reverse Proxy	Acts as a load balancer and reverse proxy, distributing requests across backend instances.

Task 3: GitHub Repository and Java Project Setup

A GitHub repository was created and a Spring Boot Java project (Maven, Java 21) was configured in Visual Studio Code and pushed to it. The repository is hosted at github.com/srikrishnadeveloper/System_Desgin_Lab_Team.

1. Created a new repository on GitHub named System_Desgin_Lab_Team with an initial README.
2. Generated a Spring Boot project (Maven, Java 21, spring-boot-starter-webmvc, spring-boot-starter-data-mongodb, Lombok) via Spring Initializr and opened it in VS Code.
3. Added a standalone class (project/Main.java) that prints the team's names.
4. Initialized Git in the project folder, connected it to the GitHub repository, and pushed the project.

project/Main.java

```
package com.systemdesign.lab;

public class Main {

    public static void main(String[] args) {
        System.out.println("team 42 - system design lab");
        System.out.println();
        System.out.println("dinesh kumar j");
        System.out.println("gopinath s");
        System.out.println("harish r");
        System.out.println("srikrishna o s");
    }
}
```

Tasks Performed

S.No	Task Performed	Observations / Findings
1	Studied System Design Fundamentals	Reviewed scalability, availability, reliability, consistency, latency/throughput, and the CAP theorem.
2	Studied Laboratory Software Tools	Reviewed installation, configuration, and purpose of VS Code, Java, JMeter, Postman, MongoDB, Kafka, GitHub, Redis, and Nginx.
3	Prepared Tool Comparison Table	Documented each tool's category and role in building scalable distributed systems.

S.No	Task Performed	Observations / Findings
4	Created GitHub Repository	Created and pushed to github.com/srikrishnadeveloper/System_Desgin_Lab_Team.
5	Configured Spring Boot Java Project	Set up a Maven-based Spring Boot 4.0.7 project (Java 21) in VS Code with WebMVC, MongoDB, and Lombok dependencies.
6	Added project/Main.java	Wrote a small class that prints the team's names.

Learning Outcomes

After completing this experiment, we were able to:

- Explain the fundamentals of system design, including scalability, availability, and consistency.
- Describe the role of major components (load balancers, caches, databases, message queues) in scalable distributed systems.
- Identify the purpose of each software tool used in the laboratory.
- Set up the development environment (VS Code, Java, GitHub) required for implementing system design experiments.